SMARTBRIDGE
Let's Bridge the Gap

SKILL WALLET
A SmartBridge Product

Project Documentation

Emotion Detection and Learning Support Engine

Date	15 July 2026
Team ID	XXXX
Project Name	Emotion Detection and Learning Support Engine

Note: This project is built with Python, Streamlit, TensorFlow/Keras, and PyTorch - not the MERN stack (React / Node.js / Express / MongoDB) this document template was originally designed for. The section structure below follows the same template, with each section describing this project's actual architecture instead.

1. Introduction

Project Title: Emotion Detection and Learning Support Engine

Team Members: Tejaswini - SmartBridge Internship Project (solo development: architecture, model integration, application development, and documentation)

2. Project Overview

Purpose:

To detect a student's emotional state (Bored, Confident, Confused, Curious, Frustrated) from free-text descriptions of their learning challenges, and generate a personalized, emotion-aware response that supports their learning in the moment - rather than treating every student the same regardless of how they actually feel.

Features:

- Dual-model emotion detection using an independently trained BiLSTM (TensorFlow/Keras) and a fine-tuned BERT (PyTorch/HuggingFace Transformers), run side-by-side for comparison.
- Keyword-based boosting layer that sharpens ambiguous model predictions.
- Mixed-emotion detection (e.g. "Bored + Frustrated") instead of a single forced label.
- Personalized, field-specific responses generated via the Google Gemini API, with an automatic template-based fallback.
- Session analytics dashboard (Plotly) showing emotion distribution and trends over time.
- Continuous interaction logging to CSV for future analysis or retraining.

3. Architecture

Frontend:

Built entirely with **Streamlit** rather than React. The UI is defined directly in Python (app.py): a field selector, a problem-description text area, quick-example prompt buttons, an emotion/response display area, and a tabbed Plotly analytics dashboard. Streamlit re-runs the script top-to-bottom on each interaction, with @st.cache_resource used to avoid reloading the ML models on every rerun.

Backend:

There is no separate Node.js/Express server. All "backend" logic - text preprocessing, dual-model inference, keyword boosting, and Gemini API orchestration - runs inside the same Python process as the Streamlit frontend, organized into a src/ package: src/preprocessing.py (TextPreprocessor), src/model.py (BiLSTMModel), src/bert_model.py (BERTEmotionClassifier), and src/predict.py (EmotionPredictor, keyword boosting).

Database:

There is no MongoDB or any database engine. Since the app has no login system, interaction data is instead appended to two flat CSV files via Pandas:

File	Columns	Purpose
emotion_response_examples.csv	text, emotion, confidence, response, field, timestamp	One row per interaction - full log for future analysis/retraining
emotion_response_mapping.csv	emotion, response	First response generated per unique emotion label

4. Setup Instructions

Prerequisites:

- Python 3.9 or higher
- pip (Python package manager)
- Trained model files under models/bltsm/ and models/bert_emotion_model_final/
- A Google Gemini API key (optional - the app falls back to template responses without one)

Installation:

```
git clone https://github.com/124teja/Emotion-Detection-Learning-Support-Engine
cd Emotion-Detection-Learning-Support-Engine
python -m venv .venv
.venv\Scripts\activate (Windows) or source .venv/bin/activate (macOS/Linux)
pip install -r requirements.txt
```

Create a .env file in the project root with:

```
GEMINI_API_KEY=your_key_here
```

5. Folder Structure

This project does not have a client/server split like a MERN app. Instead, it has a single Python application with a small internal package:

```
Emotion-Detection-Learning-Support-Engine/
■■■ app.py                          # Streamlit UI + orchestration (the "frontend + backend")
■■■ src/
■   ■■■ preprocessing.py            # TextPreprocessor.clean_text()
■   ■■■ model.py                    # BiLSTMModel (Keras)
■   ■■■ bert_model.py               # BERTEmotionClassifier (PyTorch)
■   ■■■ predict.py                  # EmotionPredictor (keyword boosting)
■■■ models/
■   ■■■ bltsm/                      # BiLSTM model, tokenizer, label encoder
■   ■■■ bert_emotion_model_final/   # Fine-tuned BERT model, tokenizer, config
■■■ emotion_response_examples.csv
■■■ emotion_response_mapping.csv
■■■ .env                            # GEMINI_API_KEY (not committed)
■■■ requirements.txt
■■■ README.md
```

6. Running the Application

Unlike a MERN app, there is only one process to start - Streamlit serves both the UI and the application logic together:

```
streamlit run app.py
```

This opens the app locally (typically at http://localhost:8501).

7. API Documentation

There is no exposed REST API in this project - Streamlit calls Python functions directly within the same process. The table below documents the key internal methods that act as the application's "API surface" instead.

Method	Parameters	Returns
TextPreprocessor.clean_text(text)	text: str	Cleaned/tokenized string
BiLSTMModel.predict_proba(cleaned_text)	cleaned_text: str	NumPy array of 5 softmax probabilities
EmotionPredictor.boost_with_keywords(text, probs)	text: str, probs: array	Keyword-boosted probability array
BERTEmotionClassifier.predict(text)	text: str	dict: emotion, confidence, scores, cleaned_text
get_gemini_response(field, problem, emotion, confidence)	field: str, problem: str, emotion: str, confidence: float	Generated response string, or None on failure (triggers template fallback)

8. Authentication

This application has **no authentication or authorization system** - there are no user accounts, tokens, or sessions tied to an identity. It is a single-session, open-access tool: any visitor can immediately use the app without signing in, and no personally identifiable information is collected or stored. This was a deliberate scope decision to keep the tool frictionless and privacy-friendly rather than an oversight.

9. User Interface

[Insert screenshots here of: the main input screen (field selector + problem text area), the dual-model comparison view (BiLSTM vs BERT with mixed-emotion display), the AI response panel, and the analytics dashboard tabs (Emotions / Fields / Summary).]

10. Testing

Testing was primarily manual, combined with small standalone diagnostic scripts written during code review to isolate specific bugs (e.g. a script that called BERTEmotionClassifier.predict_proba() directly, bypassing all keyword logic, to confirm whether over-confident predictions originated from the base model or from the inference-layer weighting). Testing covered:

- Cross-browser verification - consistent layout, widgets, and charts across browsers.
- Performance and caching - confirming models load only once per session via @st.cache_resource, with stable response times across repeated interactions.
- Environment and final validation - correct environment variables, required model files present, full workflow functioning end-to-end.
- Targeted regression checks after each bug fix (softmax normalization, keyword collision, class-weight bias) using hand-picked test sentences with known expected outcomes.

11. Screenshots or Demo

Demo video: https://drive.google.com/file/d/172zIj9gy3DOIl11TIeuo2cJLiqBDTdf5/view?usp=sharing
GitHub repository: https://github.com/124teja/Emotion-Detection-Learning-Support-Engine
[Insert additional screenshots here if not already covered in Section 9.]

12. Known Issues

- BERT produces very high-confidence predictions (often 95%+) on in-domain text, which means it rarely surfaces a second "mixed" emotion above the confidence threshold used for BiLSTM - a genuine architectural difference, not a bug.
- No "Neutral"/"Other" class - out-of-domain or unrelated input (e.g. small talk, gibberish) is still forced into one of the 5 emotion labels, sometimes with misleadingly high confidence.
- Keyword-boosting is rule-based, not learned, and can be fooled by phrasing outside its curated keyword lists.
- No load balancing or redundancy - the app currently runs as a single instance with no failover if that instance goes down (aside from the Gemini-to-template fallback for API failures specifically).

13. Future Enhancements

- Retrain with an added "Neutral"/"Other" class to improve out-of-domain robustness.
- Tune the mixed-emotion confidence threshold separately per model, given their differing probability distributions.
- Add multilingual emotion detection support.
- Introduce optional persistent user accounts to track a learner's emotional trend across multiple sessions, not just within one.
- Deploy to a cloud platform (Streamlit Community Cloud or Hugging Face Spaces) for public access.
- Expand and refine the keyword-boosting lists using real logged data from emotion_response_examples.csv.